Learning Heuristics for the Variable Gapped Longest Common Subsequence Problem

1 Introduction

This work proposes a learning framework for automatic constructing a data-driven heuristic to guide Beam Search in combinatorial optimization problems, specifically demonstrated for a sequence problem that arises in bioinformatics, the *Variable Gapped Longest Common Subsequence Problem* (VGLCSP). A feed-forward neural network (FFNN) is used to parameterize the heuristic function, in particular as its outcome provide through the feed-forward pass. Instead of relying on gradient-based training which is mainly used in the context of (constrained) continuous problems minimizing the error in an iterative procedure by using numerical methods such as gradient descent, the network weights are optimized here using effective (population-based) meta-heuristics such as Genetic Algorithm (GA), precisely its popular version called the biased RKGA (BRKGA) approach.

Each individual in the GA population represents a complete set of neural network weights. The quality of an individual is evaluated by embedding the neural network inside the *Iterative Multi-source Beam Search* (IMSBS) framework (proposed in the paper for EUROCAST 2026) to guide the search in the process of forward passes of the BS, and measuring the average solution quality obtained on a set of training instances. Validation instances are used exclusively to monitor generalization.

Formally, the learning problem is defined as:

$$\max_{\theta} \frac{1}{|T|} \sum_{I \in T} f(\text{IMS_BeamSearch}(I, h_\theta)),$$

where T denotes the training set, h_θ is the neural heuristic parameterized by weights θ, and $f(\cdot)$ measures solution quality.

2 Neural Network Model

The heuristic is represented by a *multilayer perceptron* with an arbitrary number of hidden layers and user-defined units per layer. Supported activation functions include `hyperbolic tangent`, `ReLU`, and `sigmoid`.

For each layer l, forward propagation is defined as:

$$\mathbf{z}^{(l+1)} = W^{(l)}\mathbf{a}^{(l)} + b^{(l)}, \quad \mathbf{a}^{(l+1)} = \phi(\mathbf{z}^{(l+1)}),$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector, respectively, and ϕ denotes the activation function.

All weights and biases are flattened into a single vector, which forms the chromosome of a GA individual.

2.1 Fitness Evaluation

Given a candidate weight vector:

1. The weights are loaded into the FFNN.
2. IMSBS is executed on each training instance using the neural heuristic (guiding the search in the forward iterations as more sensible than the backward iterations).
3. The size of the best sequence obtained for each instance is recorded.

The fitness value is computed as the average solution quality over all training instances. Validation performance is computed analogously on a separate dataset and is used only for reporting.

(Q0) Speed up – evaluate fitness value by using parallelization.

2.2 Feature Extraction for the Variable-Gap LCS

To guide the search process, each partial solution (node) is represented by a fixed-length numerical feature vector capturing both the current state of the construction and the structure of the remaining search space. These features are designed to be instance-independent and comparable across problem sizes.

Let $S = \{s_1, \ldots, s_k\}$ denote the set of input sequences, and let p_i be the current matched position in sequence s_i at a given node. The vector $\mathbf{p}^L = (p_1, \ldots, p_k)$ is first normalized by the corresponding sequence lengths,

$$p_i \leftarrow \frac{p_i}{|s_i|},$$

so that progress in each sequence lies in $[0, 1]$ and becomes independent of absolute input sizes.

From this normalized position vector, four statistical descriptors are extracted: maximum, minimum, mean, and standard deviation (4 features). These quantities characterize, respectively, the most advanced and least advanced sequences, the average depth of the search, and the imbalance of progress among sequences. In addition, *the current length of the partially constructed subsequence* is included as a feature (as the fifth feature). Together, these values provide a compact description of the current search state.

To capture the feasibility of future extensions, gap-related features are computed from the positions of remaining sequences (subproblem) of input sequences. For each sequence s_i, all gap bounds associated with positions strictly after the current node position are collected. Symbolically, for each v, Gap_i^{min} is calculated as $\min\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, Gap_i^{max} is calculated as $\max\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, Gap_i^{avg} as $\mathrm{AVG}\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, and Gap_i^{std} as $\mathrm{STD}\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, where $G_1, \ldots, G_m$ are gap constraints associated to $s_1, \ldots, s_m$ respectively. This gives us additionally 4 features; in overall 9 features are used for the **feature configuration 1** (F1). These statistics for gap constrains summarize the remaining flexibility of the instance: large average or maximum gaps indicate greater freedom for future matches, whereas small minimum gaps and large variability typically signal tighter constraints and increased difficulty. Additional feature $|\Sigma|$, this 10 features, created the **feature configuration 2** (F2); When adding m (number of input sequences) to F2, we obtain **feature configuration 3** (F3).

The alphabet size $|\Sigma|$ and the number of sequences (m) as the attributes provide global structural information about the instance and help the learning model adapt to varying problem scales.

Finally, all features are standardized to zero mean and unit variance prior to being passed to the learning component. The utilization of normalization is know to contribute decreased sensibility of learning process when ML models are trained. Overall, the resulting representation combines information about (i) the current construction progress, (ii) the remaining gap-constraint

landscape, and (iii) global instance properties, yielding between 9 and 11 features depending on the configuration. This compact yet expressive encoding enables effective learning-based guidance of the search for high-quality variable-gap longest common subsequences.

3 Genetic Algorithm

The optimization process follows a population-based evolutionary scheme.

3.1 Initialization

Each individual is initialized by sampling weights uniformly from a predefined interval $[-w, w]$ (the value w parametrized, but fixed to 1).

3.2 Elitism

A fixed number of top-performing individuals are copied unchanged into the next generation.

3.3 Mutation

Several individuals are generated completely at random (random immigrants). This mechanism promotes exploration and maintains population diversity.

3.4 Offspring Generation

The best parent-selection strategy chosen:

- **BRKGA**: One parent is selected from the elite set and one from the non-elite set. Each gene is inherited from the elite parent with a predefined probability.

We opt to use the *BRKGA* as the first option to work well.

3.5 Replacement and Termination

The new population consists of elites, mutants, and offspring. The algorithm terminates once a predefined time limit is reached (in this case we used 2 hours). The best individual encountered during training is continuously tracked and stored. We employed an early termination by monitoring the validation value. In details, if a new incumbent configuration has reached, we determine the validation quality. If the value is decreased acc. to the best so far validation quality, the process of learning terminates by reporting overfitting.

4 Overall Learning Procedure

The learning process alternates between evolutionary optimization of network weights and evaluation through (iterative multi-source) Beam Search. Importantly, the neural network does not directly predict solutions; instead, it learns how to *guide* the search procedure. This places the method within the class of neuro-evolutionary hyper-heuristics.

5 Algorithm

Algorithm 1 GA-Based Learning of a Beam Search Heuristic (BRKGA)

```
Initialize population P with random neural network weights
Evaluate all individuals on training set T
Store best solution θ*
start ← current time
while time limit T_max has not exceeded do
    Sort P by fitness (descending) // Section 2.1
    Initialize empty population P'
    Copy top n_elites individuals from P to P'
    for i = 1 to n_mutants do
        Create random individual x
        Evaluate x on T
        Add x to P'
        Update θ* if improved
    end for
    for i = 1 to n_offspring do
        Select parents using, BRKGA
        Generate offspring by crossover
        Evaluate offspring on T
        Add offspring to P'
        Update θ* if improved
    end for
    P ← P'
end while
return best weights θ*
```

As the best optimizer, the BRKGA setting used in training: $n_{pop} = 20, n_{elites} = 1, p_e = 0.5, p_m = 7$.

5.1 Discussion

The proposed framework learns heuristics rather than explicit solutions. The neural network is optimized exclusively through downstream search performance, enabling learning in non-differentiable settings. This tightly coupled integration of evolutionary learning and the classical optimization algorithm Beam Search allows the system to adapt its guidance strategy to the structure of the problem instances.

5.2 Improvements: an ensemble method

To guide the beam search, we employ a rank-based ensemble strategy that combines a learned heuristic with an analytical upper bound (UB) estimate. Specifically, each candidate node is evaluated using two criteria: (i) a neural score obtained from a trained feed-forward neural network (FFNN), and (ii) a problem-specific upper bound (UB_2 in the EUROCAST paper).

Instead of combining the raw scores directly, which may differ in scale and distribution, we adopt a rank aggregation approach. First, candidate nodes are sorted independently according to each criterion, producing two rankings. Each node is then assigned a rank based on its position in each sorted list.

The final ranking score is computed as a weighted combination of the two ranks:

$$r(v) = \alpha \cdot r_{\mathrm{FFNN}}(v) + (1 - \alpha) \cdot r_{\mathrm{UB}}(v),$$

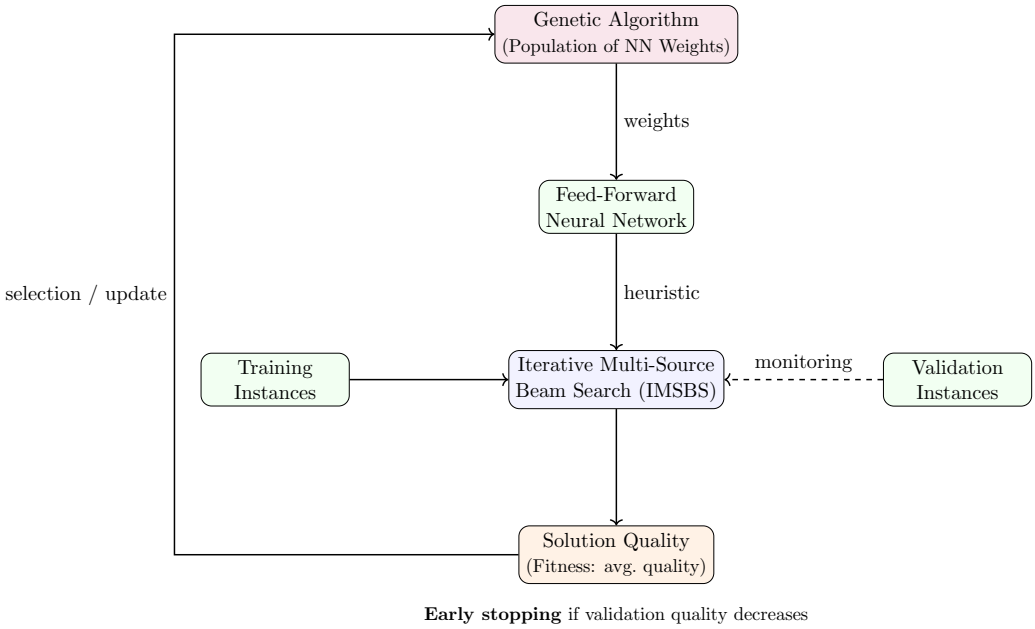

Figure 1: Enhanced learning framework integrating a Genetic Algorithm with IMSBS. The GA evolves neural network weights, which define a heuristic used within the beam search. Training instances guide optimization, while validation instances monitor generalization. The feedback loop updates the population based on solution quality.

where $\alpha = 0.45$ in our implementation except for $m = 3$ where $\alpha = 0.5$ is fixed. Nodes are then sorted in ascending order of $r(v)$, and the top candidates are retained according to the beam width.

This rank-based ensemble provides a robust mechanism for integrating learned and analytical guidance, mitigating scale inconsistencies and improving search stability.

6 Preliminary results

IMSBS setting (tuned by `irace`, values of two parameters tuned: β and $\#imbs_iters = 3000$); other parameters are less sensible and fixed according to EUROCAST 2026 paper study:

- $\beta = 2000$, $h' = \mathrm{UB}_2$, $\#imbs_iters = 5000, \#num_roots = 10$

The Learning IMSBS algorithm setting as that used in IMSBS, to be completely fair in the comparisons with the number of performed iterations. The neural network configuration in this case, obtained after running several 2-layer (n_1, n_2) and 3-layer configurations (n_1, n_2, n_3) for $n_1, n_2, n_3 \in \{5, 10\}$ (neurons) and $n_1 \geq n_2 \geq n_3$ splitting instances per each Σ. We set up the same activation function in all layers choosing among the three (most) known $\{\texttt{tanh}, \texttt{ReLU}, \texttt{sigmoid}\}$.

We have split the random instances into 4 different subsets, according to different m, and run a GA process on each pf these subsets independently. In that way, one configuration per each m has been obtained after the training process. In details, the following FFNN configurations are obtained:

- $m = 2$: units in FFNN: (10, 5, 5), activation function: `ReLu` and the feature set (F2);
- $m = 3$: units in FFNN (10, 10, 5), activation function: `ReLu` and the feature set (F2);
- $m = 5$: units in FFNN (10, 5, 5), activation function `sigmoid`, the feature set (F1);
- $m = 10$: units in FFNN (10, 5, 5), activation function `ReLu`, the feature set (F2).

Table 1: Comparison of objective values between IMSBS and LIMSBS. Best results per instance category are in bold.

m	n	$\|\Sigma\|$	LMSBS-ensemble	IMSBS
2	100	2	72.4	**72.9**
2	100	4	**62.0**	61.9
2	200	2	**142.5**	139.3
2	200	4	**125.9**	125.5
2	50	2	37.3	**37.7**
2	50	4	**30.1**	**30.1**
2	500	2	**305.9**	282.1
2	500	4	**305.5**	303.0
3	100	2	**58.5**	**58.5**
3	100	4	**48.7**	48.5
3	200	2	**106.7**	106.4
3	200	4	**97.9**	97.8
3	50	2	**31.2**	**31.2**
3	50	4	**22.9**	**22.9**
3	500	2	**154.9**	150.6
3	500	4	**237.6**	236.0
Avg.			**115.0**	112.8

m	n	$\|\Sigma\|$	LIMSBS-ensemble	IMSBS
5	100	2	32.6	**32.9**
5	100	4	28.0	**28.1**
5	200	2	**45.5**	42.1
5	200	4	**41.8**	38.8
5	50	2	**20.5**	**20.5**
5	50	4	**15.3**	**15.3**
5	500	2	**45.7**	39.2
5	500	4	**58.0**	45.6
10	100	2	**14.3**	12.0
10	100	4	**9.6**	8.9
10	200	2	**14.3**	12.8
10	200	4	**9.4**	8.2
10	50	2	**11.7**	**11.7**
10	50	4	7.3	**7.5**
10	500	2	**14.3**	13.2
10	500	4	**10.0**	8.6
Avg.			**23.6**	21.6

Table 2: Statistical comparison between LIMSBS and IMSBS using the Wilcoxon signed-rank test (one-sided).

Comparison	**Wins / Ties / Losses**	**p-value**	**Conclusion**
All instances ($m = 2, 3, 5, 10$)	20 / 7 / 5	0.009	LIMSBS significantly better
Small instances ($m = 2, 3$)	10 / 4 / 2	< 0.01	Significant difference
Large instances ($m = 5, 10$)	10 / 3 / 3	< 0.05	LIMSBS significantly better

6.1 Statistical Analysis of Solution Quality: benchmark set RANDOM

We evaluated the difference in solution quality between LIMSBS and IMSBS using the Wilcoxon signed-rank test on paired instances, reported in Table 2.

The test reveals a statistically significant difference in favor of LIMSBS, indicating that LIMSBS achieves superior performance overall. Out of 32 instances, LIMSBS outperforms IMSBS in 20 cases, while 7 instances result in ties and 5 favor IMSBS.

These results demonstrate that the proposed approach provides both statistically and practically significant gains over IMSBS.

7 Practically-motivated VGLCSP instances

What we are really modeling is: context-dependent importance of positions. This aligns nicely with:

- sequence alignment scoring
- attention mechanisms
- weighted subsequence problems

Interpretation (very important) More negative = more stable = stronger influence ... Stability ranking picked from literature (provide a reference):

- $GC > CG > GG > GA \approx GT \approx CA > CT > AA > AT > TAGC > CG > GG > GA \approx GT \approx CA > CT > AA > AT > TA$

This exact trend is experimentally confirmed.

Energy-aware LCS / weighted subsequence. TODO

$DINUC_WEIGHT = \{'AA' : 1.00,'TT' : 1.00,'AT' : 0.88,'TA' : 0.58,'CA' : 1.45,'TG' : 1.45,'GT' : 1.44,'AC' : 1.44,'CT' : 1.28,'AG' : 1.28,'GA' : 1.30,'TC' : 1.30,'CG' : 2.17,'GC' : 2.24,'GG' : 1.84,'CC' : 1.84\}$

Structure-aware variable-gap LCS... Where: constraints are data-driven.

We state: "*Gap constraints are proportional to local thermodynamic stability estimated via nearest-neighbor interactions.*"

7.1 Structure-Aware Benchmark Generator for Multi-Sequence Gapped LCS

We propose a benchmark generator for the multi-sequence gapped longest common subsequence (GLCS) problem, where position-dependent gap constraints are derived from local structural influence.

Given a set of sequences: $\mathcal{S} = \{S_1, S_2, \ldots, S_m\}$, each sequence $S_j = (s_j[1], \ldots, s_j[n])$ is associated with gap constraints: $w^{(j)} = (w_1^{(j)}, \ldots, w_n^{(j)})$, where $w_i^{(j)}$ determines the maximum number of symbols that may be skipped before matching position i.

7.2 Dinucleotide-Based Structural Model

We define a dinucleotide weight function:

$$f(s_i, s_{i+1}) = -\Delta G(s_i, s_{i+1}),$$

where ΔG denotes the nearest-neighbor free energy.

Causal positional contribution. To ensure consistency with the directional nature of the gapped LCS constraint, we define a *causal* positional contribution. Each position k is associated only with the interaction formed with its immediate predecessor:

$$f_k^{(j)} = \begin{cases} f(s_{k-1}^{(j)}, s_k^{(j)}), & k > 1, \\ 0, & k = 1. \end{cases}$$

Interpretation.

- Each position inherits structural influence from the bond linking it to the previous nucleotide (linkage between these considered).
- First position has no predecessor and is therefore assigned zero contribution.

Rationale. This definition enforces a left-to-right dependency structure, ensuring that all subsequent computations of influence and gap constraints depend only on previously observed positions. This is consistent with the feasibility condition in the gapped LCS problem, where admissibility of extending a subsequence depends on the last selected position.

Remark. Compared to centered models, the causal formulation removes any dependence on future positions, resulting in a strictly sequential and history-aware structural representation.

7.3 Causal (Left-Sided) Spatial Influence Aggregation

In order to align the influence model with the semantics of the gapped LCS constraint, we define influence in a *causal* (left-to-right) manner.

The influence at position i is computed using only preceding positions:

$$I^{(j)}(i) = \sum_{k=\max(1,i-r)}^{i} e^{-\lambda(i-k)} \cdot f_k^{(j)}.$$

Interpretation.

- Only positions $k \leq i$ contribute to the influence of i.
- The exponential kernel assigns higher weight to positions closer to i.
- The parameter r limits the historical window.

Rationale. This formulation reflects the fact that, in the gapped LCS problem, feasibility of selecting position i depends on the previously selected position. Therefore, the structural context relevant to i must be derived exclusively from preceding positions.

Consequence. The resulting influence values induce *directional structure* in the sequence:

- regions preceded by strong interactions accumulate high influence,
- regions following weak or irregular segments exhibit lower influence.

Normalization.

$$\hat{I}_i^{(j)} = \frac{I_i^{(j)} - \min(I^{(j)})}{\max(I^{(j)}) - \min(I^{(j)}) + \epsilon} \in [0, 1]\,.$$

7.4 Direct Mapping to Gap Constraints

Gap constraints are obtained via a *direct mapping*:

$$w_i^{(j)} = w_{\min} + (w_{\max} - w_{\min}) \cdot \hat{I}_i^{(j)}.$$

Interpretation.

- high influence $\Rightarrow$ large allowed gap,
- low influence $\Rightarrow$ restrictive gap.

This mapping reflects the idea that positions with strong structural connectivity can support longer-range dependencies and allow more flexible matching.

7.5 Noise Injection

$w_i^{(j)} \leftarrow w_i^{(j)} + \mathcal{N}(0, \sigma^2)$, followed by clipping to $[w_{\min}, w_{\max}]$.

7.6 Resulting Instance

The generator produces: $\{(S^{(j)}, w^{(j)}) \mid j = 1, \ldots, m\}$.

Discussion. The proposed generator creates structured instances where:

- high-influence regions act as *flexible hubs*,
- low-influence regions form *bottlenecks*.

This leads to heterogeneous constraint landscapes, where solution feasibility depends on navigating between restrictive and permissive regions.

Such instances are particularly challenging for exact and heuristic algorithms, as they combine large feasible regions with critical bottlenecks.

7.7 Difficulty Control Parameters

- m: number of sequences
- r: influence radius
- λ: decay factor
- $[w_{\min}, w_{\max}]$: gap bounds
- σ: noise level

Conclusion. By linking gap constraints directly to structural influence, the generator produces instances with controllable heterogeneity and realistic dependency patterns. The resulting benchmarks enable systematic evaluation of multi-sequence gapped LCS algorithms under diverse and challenging conditions.

7.8 Results on RAT & VIRUS benchmark set

To demonstrate effectiveness of the two approaches on real-world instances, we have generated a set of instances by taking well-known benchmark sets Rat and Virus from the LCS problem literature. In details, we consider only those instances with $|\Sigma| = 4$, as for the instances with $|\Sigma| = 20$, the results were rather short solutions—therefore not relevant from the biological standpoint— generating gaps in the same way as for the synthetic instances. On each input sequences in each instances, we have generated one gap constraint by the methodology introduced in this chapter. In this way, input sequences are copied in corresponding VGLCS instances, while gap constraints have been accompanied after generation. This dataset consists of $2 \cdot 10 = 20$ problem instances. All sequences are of length 600, and, as already mentioned, over an alphabet set of size 4. Their differ characteristically by m values which range from 10 to 200. In our experiments, we have taken the settings of Imsbs used in for synthetic instances, and for the Limsbs-ensemble, a neural network trained specifically for a largest our m because in the case of these 20 instances, $m \geq 10$ holds.

The following conclusions can be made form the obtained results, reprted in Table 3. The results indicate a clear advantage of LIMSBS in terms of solution quality.

- Overall, LIMSBS outperforms IMSBS in 15 out of 20 instances, while 5 instances result in ties and none favor IMSBS. This demonstrates a consistent dominance of LIMSBS across the tested dataset.
- In terms of solution quality, LIMSBS achieves an average value of 11.8, compared to 9.9 for IMSBS, corresponding to an average relative improvement of approximately 19%. The median quality values are 5 (LIMSBS) and 4 (IMSBS), confirming that improvements are consistent and not driven by outliers.
- The advantage of LIMSBS is particularly pronounced on more difficult instances (e.g., virus cases with higher variability), where substantial gains are observed (e.g., 87 vs. 36). In contrast, ties typically occur on simpler instances with very low objective values.
- Regarding computational time, IMSBS is generally faster, achieving an average runtime of 36.9 seconds compared to 32.6 seconds for LIMSBS. However, this difference is not consistent across all instances, and in several cases LIMSBS is competitive or even faster.

The results again demonstrate that LIMSBS provides a substantial and consistent improvement in solution quality over IMSBS on real-world instances, at the cost of a comparable computational effort. This makes LIMSBS a preferable choice when solution quality is the primary objective.

Table 3: Comparison of LIMSBS and IMSBS on real instances.

n	type	quality_limsbs	time_limsbs	quality_imsbs	time_imsbs
4	rat	**1**	25.71	**1**	44.30
4	virus	**3**	35.27	**3**	43.25
4	rat	**28**	43.44	25	30.65
4	virus	**87**	134.51	36	43.14
4	rat	**1**	39.51	**1**	66.57
4	virus	**4**	27.71	3	31.80
4	rat	**11**	23.39	**11**	24.26
4	virus	**19**	27.08	**19**	36.52
4	rat	**1**	39.82	**1**	62.54
4	virus	**1**	32.06	**1**	48.59
4	rat	**10**	11.31	9	49.36
4	virus	**11**	16.31	10	26.92
4	rat	**9**	17.53	7	29.62
4	virus	**13**	16.66	9	30.40
4	rat	**5**	18.18	4	48.64
4	virus	**7**	20.39	5	32.12
4	rat	**3**	22.62	1	45.22
4	virus	**5**	20.26	4	34.38
4	rat	**1**	22.52	**1**	53.02
4	virus	**4**	22.80	3	40.64

8 Future Work

The training process. Using only the mean optimizes average performance, ignores failures, and can produce unstable heuristics. In COPs, robustness is often more important than average performance.

Fitness function replacement:

- Option A (robust & simple) Fitness = 0.7 · mean + 0.3 · 20th-percentile
- Option B (robust learning): Fitness = 20-th percentile
- allow some degradation in the validation fitness score, not immediately stop by obtaining a weaker score.

Curriculum / weighted fitness. Give more importance to harder instances and recent failures. Fitness=$\sum_i w_i f_i$, e.g., $w_i = \frac{1}{2^i}$, or $w_i = \frac{1}{n+1}$.